COMP3242/6242: Deep Learning

Week 3 — Back-propagation and Learning

Stephen Gould
stephen.gould@anu.edu.au

Australian National University

Semester 1, 2026

Overview

- review
  - supervised learning and empirical risk minimization
  - multi-layer perceptrons
- neural networks as end-to-end computation graphs
- automatic differentiation
  - forward and reverse mode
- back-propagation
  - forward and backward passes
  - exploding and vanishing gradients
  - memory layout
- learning: putting it all together
  - initialization
  - stochastic gradient descent (SGD)
  - iterations, epochs and data shuffling
  - evaluating and selecting models
  - speeding up vanilla SGD

(Supervised) Learning Review

- parametrized model $y = f(x; \theta)$ and training set $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}$
- regularized loss function L that measures how well the model fits the training data, usually decomposes over the training data plus a prior over parameters

$$L(\theta) = \sum_i \ell\left(f(x^{(i)}; \theta), y^{(i)}\right) + R(\theta)$$

- we then optimize the loss function with respect to the model parameters

$$\theta^\star = \arg\min_\theta L(\theta)$$

  - L usually non-convex in θ, so we're often happy with just a local minimum
- simplest optimization algorithm is gradient descent

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

Multilayer Perceptron (MLP)

- most basic deep learning model is the multilayer perceptron
- input $x \in \mathbb{R}^{p_0}$, hidden layers $z_j \in \mathbb{R}^{p_j}$, output $y \in \mathbb{R}^{p_n}$
- each layer computes its output as

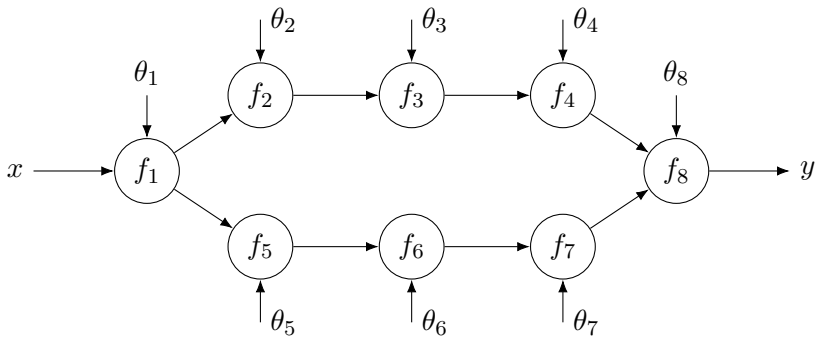
$$z_j = f_j(z_{j-1}) = \sigma_j(A_j z_{j-1} + b_j)$$

  where $A_j \in \mathbb{R}^{p_j \times p_{j-1}}$ and $b_j \in \mathbb{R}^{p_j}$ are parameters and σ_j is a non-linear (elementwise) activation function ($x \triangleq z_0$, $y \triangleq z_n$)
- the network output is then the composition

$$\begin{aligned} y &= (f_n \circ \cdots \circ f_2 \circ f_1)(x) \\ &= \sigma_n(A_n \sigma_{n-1}(\cdots \sigma_1(A_1 x + b_1)) + b_n) \end{aligned}$$

End-to-end Computation Graph

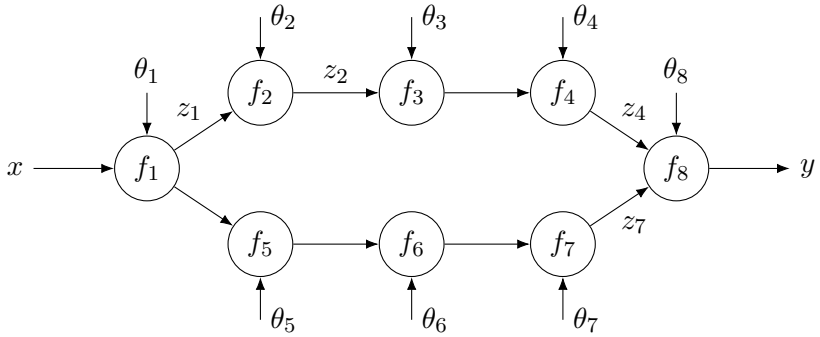
Deep learning defines a mapping from input to output (equiv. computation graph) composed of many simple parametrized functions (equiv. computation nodes).



$$y = f_8(f_4(f_3(f_2(f_1(x)))), f_7(f_6(f_5(f_1(x)))))$$

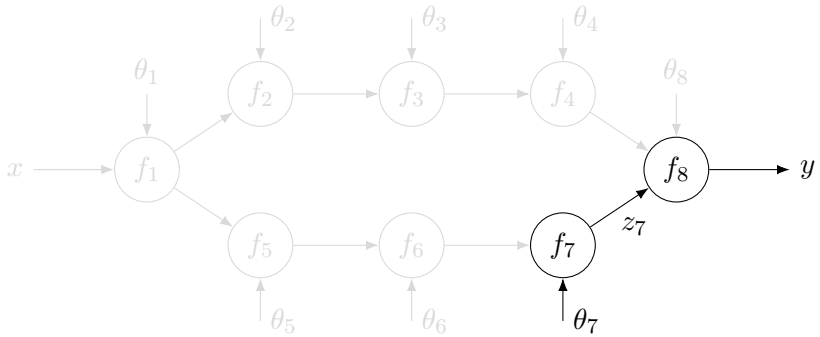
(parameters θ_i omitted for brevity)

Function Evaluation



$$
\begin{aligned}
z_1 &= f_1(x; \theta_1) \\
z_2 &= f_2(z_1; \theta_2) \\
&\vdots \\
y &= f_8(z_4, z_7; \theta_8)
\end{aligned}
$$

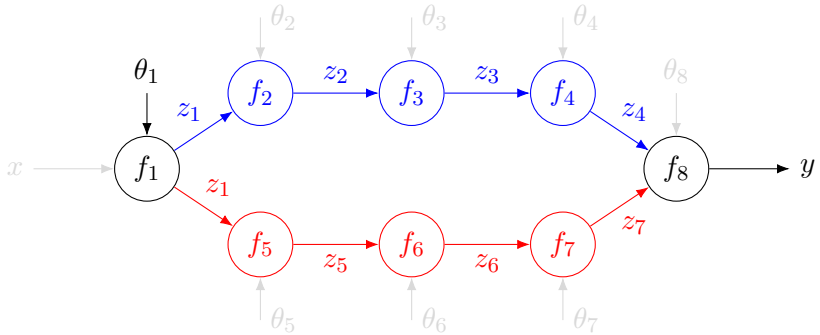
Gradient Evaluation



Example 1.

$$\frac{\partial L}{\partial \theta_7} = \frac{\partial L}{\partial y}\frac{\partial y}{\partial z_7}\frac{\partial z_7}{\partial \theta_7}$$

Gradient Evaluation



Example 2.

$$\frac{\partial L}{\partial \theta_1} = \frac{\partial L}{\partial y}\left(\frac{\partial y}{\partial z_4}\frac{\partial z_4}{\partial z_3}\frac{\partial z_3}{\partial z_2}\frac{\partial z_2}{\partial z_1} + \frac{\partial y}{\partial z_7}\frac{\partial z_7}{\partial z_6}\frac{\partial z_6}{\partial z_5}\frac{\partial z_5}{\partial z_1}\right)\frac{\partial z_1}{\partial \theta_1}$$

Notational Aside (Often Sloppy in Literature)

"the wonderful thing about standards is that there are so many of them to choose from"

For scalar-valued functions:

$$\frac{\mathrm{d}f}{\mathrm{d}x} \quad \text{or} \quad \frac{\partial f}{\partial x}$$

Notational Aside (Often Sloppy in Literature)

"the wonderful thing about standards is that there are so many of them to choose from"

For scalar-valued functions:

$$\frac{\mathrm{d}f}{\mathrm{d}x} \quad \text{or} \quad \frac{\partial f}{\partial x}$$

For multi-variate scalar-valued functions, $f : \mathbb{R}^n \to \mathbb{R}$:

$$\nabla f(x) = \left(\frac{\mathrm{d}f}{\mathrm{d}x_1}, \ldots, \frac{\mathrm{d}f}{\mathrm{d}x_n} \right) \in \mathbb{R}^n$$

Notational Aside (Often Sloppy in Literature)

"the wonderful thing about standards is that there are so many of them to choose from"

For scalar-valued functions:

$$\frac{\mathrm{d}f}{\mathrm{d}x} \qquad \text{or} \qquad \frac{\partial f}{\partial x}$$

For multi-variate scalar-valued functions, $f : \mathbb{R}^n \to \mathbb{R}$:

$$\nabla f(x) = \left(\frac{\mathrm{d}f}{\mathrm{d}x_1}, \ldots, \frac{\mathrm{d}f}{\mathrm{d}x_n} \right) \in \mathbb{R}^n$$

For multi-variate vector-valued functions, $f : \mathbb{R}^n \to \mathbb{R}^m$:

$$\frac{\mathrm{d}}{\mathrm{d}x} f(x) = \begin{bmatrix} \frac{\mathrm{d}f_1}{\mathrm{d}x_1} & \cdots & \frac{\mathrm{d}f_1}{\mathrm{d}x_n} \\ \vdots & \ddots & \vdots \\ \frac{\mathrm{d}f_m}{\mathrm{d}x_1} & \cdots & \frac{\mathrm{d}f_m}{\mathrm{d}x_n} \end{bmatrix} \in \mathbb{R}^{m \times n} \qquad \left(\frac{\partial}{\partial x} f(x, y) \text{ for partial} \right)$$

Sometimes D and D_X for $\frac{\mathrm{d}}{\mathrm{d}x}$ and $\frac{\partial}{\partial x}$, respectively.

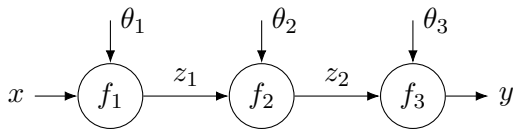
Automatic Differentiation (AD)

- algorithmic procedure that produces code for computing exact derivatives
- assumes numeric computations are composed from a small set of elementary operations that we know how to differentiate
  - arithmetic, exp, log, trigonometric
- greatly reduces development effort in modern machine learning

Automatic Differentiation (AD)

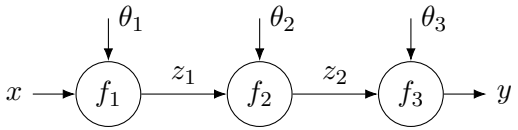
- algorithmic procedure that produces code for computing exact derivatives
- assumes numeric computations are composed from a small set of elementary operations that we know how to differentiate
  - arithmetic, exp, log, trigonometric
- greatly reduces development effort in modern machine learning
- two flavours
  - (forward mode) for a fixed independent variable u, computes derivatives $\frac{\mathrm{d}v}{\mathrm{d}u}$ for all dependent variables v
  - (reverse mode) for a fixed dependent variable v, computes derivatives $\frac{\mathrm{d}v}{\mathrm{d}u}$ for all independent variables u

Forward vs Reverse Mode Automatic Differentiation



$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}$$

Forward vs Reverse Mode Automatic Differentiation



$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}$$

Forward Mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \frac{\mathrm{d}L}{\mathrm{d}y}\left(\frac{\mathrm{d}y}{\mathrm{d}z_2}\underbrace{\left(\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}\right)}_{\mathrm{d}z_2/\mathrm{d}\theta_1}\right)$$

Reverse Mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \left(\underbrace{\left(\frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\right)}_{\mathrm{d}L/\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\right)\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}$$

Dual Numbers for Forward Mode

- forward mode gradients can be evaluated on-the-fly
- replace variable v with $v + \Delta v$
- Δv has the property that $(\Delta v)^2 = 0$
- set $\Delta u = 1$ to get $\Delta v = \frac{\mathrm{d}v}{\mathrm{d}u}$

Dual Numbers for Forward Mode

- forward mode gradients can be evaluated on-the-fly
- replace variable v with $v + \Delta v$
- Δv has the property that $(\Delta v)^2 = 0$
- set $\Delta u = 1$ to get $\Delta v = \frac{\mathrm{d}v}{\mathrm{d}u}$

Example 1.

$$y = ax + b$$

$$y + \Delta y = a(x + \Delta x) + b$$
$$\therefore \ \Delta y = a\Delta x$$

Dual Numbers for Forward Mode (Example 2)

$$y = \exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

Dual Numbers for Forward Mode (Example 2)

$$y = \exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

$$y + \Delta y = \exp(x + \Delta x)$$

Dual Numbers for Forward Mode (Example 2)

$$y = \exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

$$\begin{aligned} y + \Delta y &= \exp(x + \Delta x) \\ &= 1 + (x + \Delta x) + \frac{(x + \Delta x)^2}{2} + \frac{(x + \Delta x)^3}{3!} + \dots \end{aligned}$$

Dual Numbers for Forward Mode (Example 2)

$$y = \exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

$$\begin{aligned} y + \Delta y &= \exp(x + \Delta x) \\ &= 1 + (x + \Delta x) + \frac{(x + \Delta x)^2}{2} + \frac{(x + \Delta x)^3}{3!} + \dots \\ &= 1 + x + \Delta x + \frac{x^2 + 2x\Delta x + \Delta x^2}{2} + \frac{x^3 + 3x^2\Delta x + 3x\Delta x^2 + \Delta x^3}{3!} + \dots \end{aligned}$$

Dual Numbers for Forward Mode (Example 2)

$$y = \exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

$$\begin{aligned}
y + \Delta y &= \exp(x + \Delta x) \\
&= 1 + (x + \Delta x) + \frac{(x + \Delta x)^2}{2} + \frac{(x + \Delta x)^3}{3!} + \dots \\
&= 1 + x + \Delta x + \frac{x^2 + 2x\Delta x + \Delta x^2}{2} + \frac{x^3 + 3x^2\Delta x + 3x\Delta x^2 + \Delta x^3}{3!} + \dots \\
&= 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots \\
&\quad + \Delta x \left(1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots\right)
\end{aligned}$$

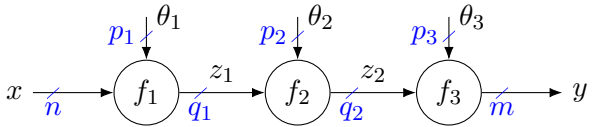
Dual Numbers for Forward Mode (Example 2)

$$y = \exp(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots$$

$$\begin{aligned}
y + \Delta y &= \exp(x + \Delta x) \\
&= 1 + (x + \Delta x) + \frac{(x + \Delta x)^2}{2} + \frac{(x + \Delta x)^3}{3!} + \dots \\
&= 1 + x + \Delta x + \frac{x^2 + 2x\Delta x + \Delta x^2}{2} + \frac{x^3 + 3x^2\Delta x + 3x\Delta x^2 + \Delta x^3}{3!} + \dots \\
&= 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots \\
&\quad + \Delta x \left(1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots\right) \\
\therefore\ \Delta y &= \Delta x \exp(x)
\end{aligned}$$

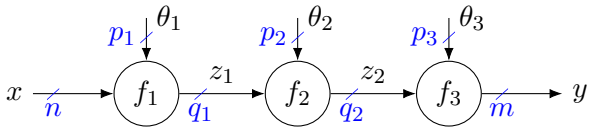
Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time



Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time

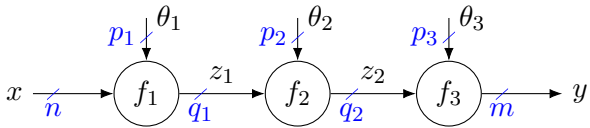


forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \frac{\mathrm{d}L}{\mathrm{d}y}\,\frac{\mathrm{d}y}{\mathrm{d}z_2}\,\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\,\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}$$

Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time

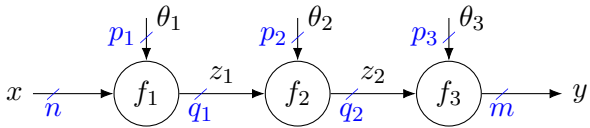


forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m}\ \underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2}\ \underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1}\ \underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}$$

Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time

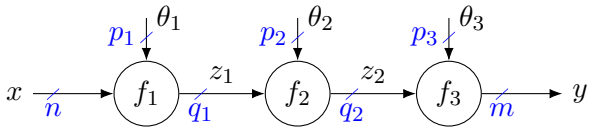


forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m} \underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2} \underbrace{\underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1} \underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}}_{q_2\times p_1}$$

Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time

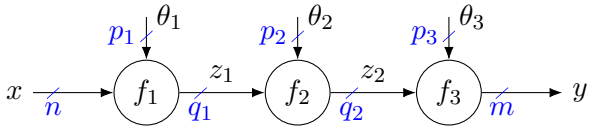


forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m} \underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2} \underbrace{\underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1} \underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}}_{q_2\times p_1}}_{m\times p_1}$$

Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time



forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m} \underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2} \underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1} \underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}$$

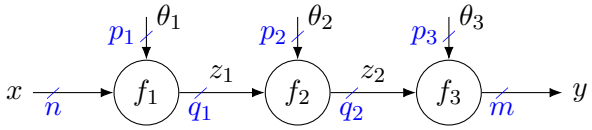
$$\underbrace{\qquad\qquad\qquad}_{q_2\times p_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad}_{m\times p_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad\qquad}_{1\times p_1}$$

Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time



forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m}\underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2}\underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1}\underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}$$

$$\underbrace{\qquad\qquad\qquad}_{q_2\times p_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad}_{m\times p_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad}_{1\times p_1}$$

reverse mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m}\underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2}\underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1}\underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}$$

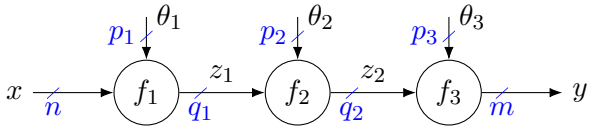
$$\underbrace{\qquad\qquad\qquad}_{1\times q_2}$$

$$\underbrace{\qquad\qquad\qquad\qquad}_{1\times q_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad}_{1\times p_1}$$

Cost of Gradient Evaluation Ordering

- in deep learning we usually want to update all parameters at the same time



forward mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta_1} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m}\underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2}\underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1}\underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}$$

$$\underbrace{\qquad\qquad}_{q_2\times p_1}$$

$$\underbrace{\qquad\qquad\qquad}_{m\times p_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad}_{1\times p_1}$$

reverse mode

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}y}}_{1\times m}\underbrace{\frac{\mathrm{d}y}{\mathrm{d}z_2}}_{m\times q_2}\underbrace{\frac{\mathrm{d}z_2}{\mathrm{d}z_1}}_{q_2\times q_1}\underbrace{\frac{\mathrm{d}z_1}{\mathrm{d}\theta_1}}_{q_1\times p_1}$$

$$\underbrace{\qquad\qquad}_{1\times q_2}$$

$$\underbrace{\qquad\qquad\qquad}_{1\times q_1}$$

$$\underbrace{\qquad\qquad\qquad\qquad}_{1\times p_1}$$

- reverse mode operations are called vector-Jacobian products
- more efficient for deep learning (scalar L, vector θ) than forward mode

Back-propagation

- reverse mode AD is called **back-propagation** in deep learning
- different deep learning frameworks use slightly different approaches (explicit graph construction versus implicit operator tracking)
- conceputally, for every line of code:

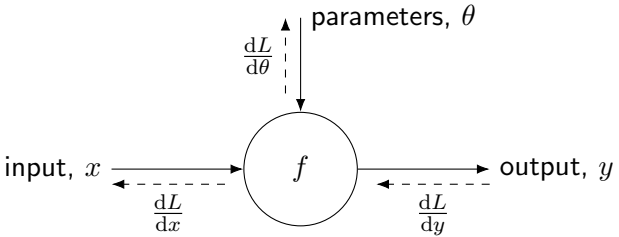
```
P, Q = foo(A, B, C)
```

  automatic differentiation produces line:

```
dLdA, dLdB, dLdC = foo_vjp(A, B, C, P, Q, dLdP, dLdQ)
```

- requires two passes through the network (one forward, one backward)

Deep Learning Node (aka Layer)



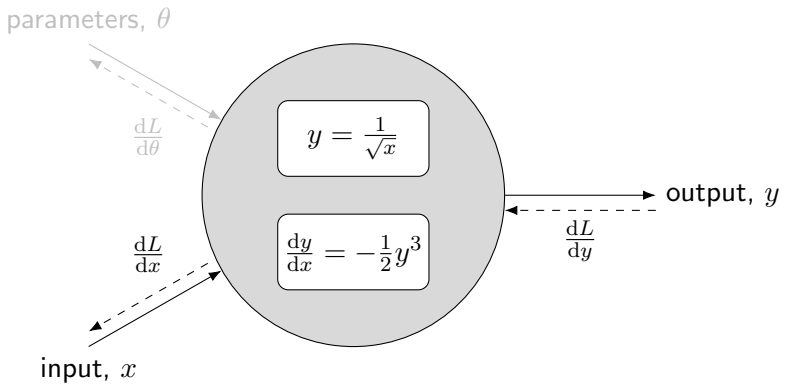
- **Forward pass:** compute output y as a function of the input x (and model parameters θ).

```
y = f(x, theta)      # evaluate function
L = loss(y, y_true) # compute loss
```

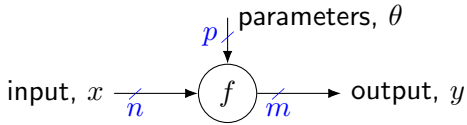
- **Backward pass:** compute the derivative of the loss with respect to the input x (and model parameters θ) given the derivative of the loss with respect to the output y.

```
L.backward() # compute all gradients
```

Example Deep Learning Node for $y = \frac{1}{\sqrt{x}}$



Batch Data and Vector-Jacobian Products



- n-dimensional input, m-dimensional output, p-dimensional parameters
- $\frac{\mathrm{d}y}{\mathrm{d}x}$ is an (m-by-n)-dimensional matrix of partial derivatives
- $\frac{\mathrm{d}y}{\mathrm{d}\theta}$ is an (m-by-p)-dimensional matrix of partial derivatives
- $\frac{\mathrm{d}L}{\mathrm{d}y}$, $\frac{\mathrm{d}L}{\mathrm{d}x}$ and $\frac{\mathrm{d}L}{\mathrm{d}p}$ are a m-, n-, and p-dimensional (row) vectors, respectively

$$\underbrace{\frac{\mathrm{d}L}{\mathrm{d}x^{(i)}}}_{1\times n} = \overbrace{\frac{\mathrm{d}L}{\mathrm{d}y^{(i)}}}^{1\times m} \underbrace{\frac{\mathrm{d}y^{(i)}}{\mathrm{d}x^{(i)}}}_{m\times n} \qquad \underbrace{\frac{\mathrm{d}L}{\mathrm{d}\theta}}_{1\times p} = \sum_{i=1}^{N} \overbrace{\frac{\mathrm{d}L}{\mathrm{d}y^{(i)}}}^{1\times m} \underbrace{\frac{\mathrm{d}y^{(i)}}{\mathrm{d}\theta}}_{m\times p}$$

- instead of forming $\frac{\mathrm{d}y}{\mathrm{d}x}$ explicitly, code can compute $\frac{\mathrm{d}L}{\mathrm{d}x}$ directly from $\frac{\mathrm{d}L}{\mathrm{d}y}$

Vanishing and Exploding Gradients

- deep learning involves long chains of gradient multiplications, e.g.,

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta}$$

Vanishing and Exploding Gradients

- deep learning involves long chains of gradient multiplications, e.g.,

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta}$$

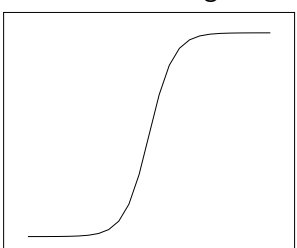
- this can result in the so-called vanishing and exploding gradient problem

Vanishing and Exploding Gradients

- deep learning involves long chains of gradient multiplications, e.g.,

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta}$$

- this can result in the so-called vanishing and exploding gradient problem
- the problem is particularly pronounced with sigmoid-like activations

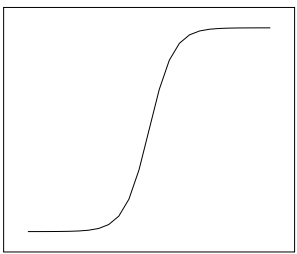


Vanishing and Exploding Gradients

- deep learning involves long chains of gradient multiplications, e.g.,

$$\frac{\mathrm{d}L}{\mathrm{d}\theta} = \frac{\mathrm{d}L}{\mathrm{d}y}\frac{\mathrm{d}y}{\mathrm{d}z_2}\frac{\mathrm{d}z_2}{\mathrm{d}z_1}\frac{\mathrm{d}z_1}{\mathrm{d}\theta}$$

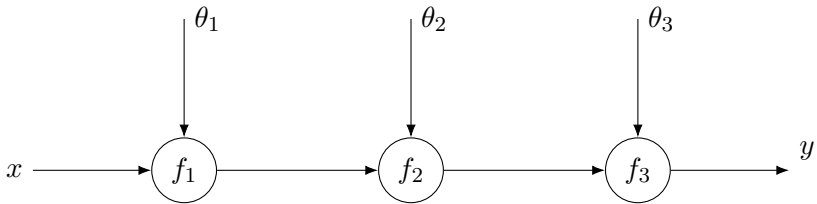
- this can result in the so-called vanishing and exploding gradient problem
- the problem is particularly pronounced with sigmoid-like activations



- a great deal of effort has been devoted to developing mitigation techniques

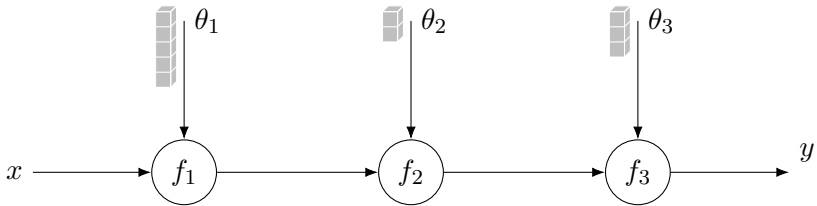
Concerning Memory

- data is often processed in batches ($B \times C \times \cdots \times N$)



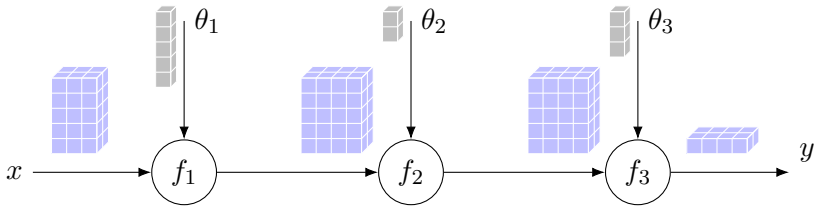
Concerning Memory

- data is often processed in batches ($B \times C \times \cdots \times N$)



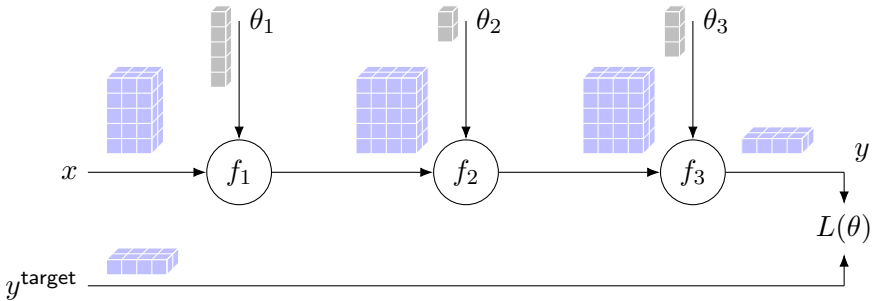
Concerning Memory

- data is often processed in batches ($B \times C \times \cdots \times N$)



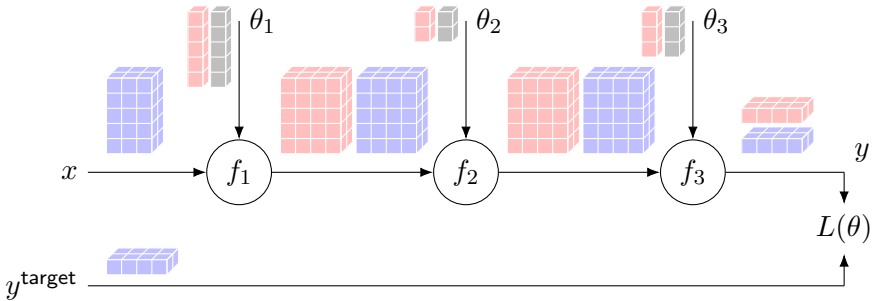
Concerning Memory

- data is often processed in batches ($B \times C \times \cdots \times N$)



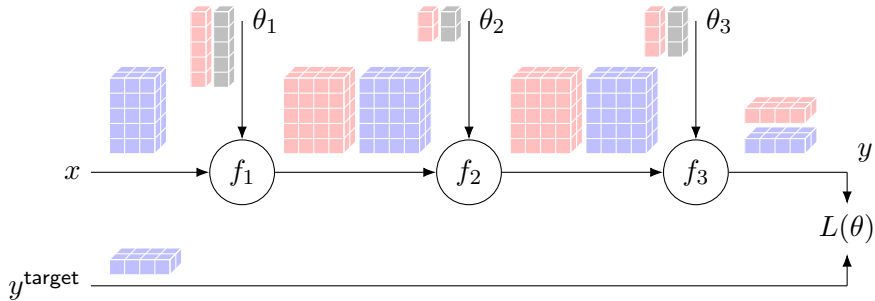
Concerning Memory

- data is often processed in batches ($B \times C \times \cdots \times N$)



Concerning Memory

- data is often processed in batches ($B \times C \times \cdots \times N$)

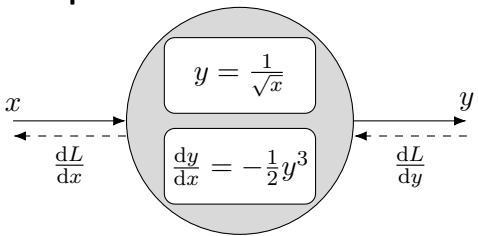


- parameters only take a small amount of memory (relative to data)
- gradients take the same amount of space as the data (stored transposed)
- in-place operations may save memory in the forward pass
- re-using buffers may save memory in the backward pass
- at test time (only forward pass) intermediate results are not stored

Python Autograd Function (Reverse Mode)

- in most cases you can compose models from existing functions
- can be extended for special cases

Example.

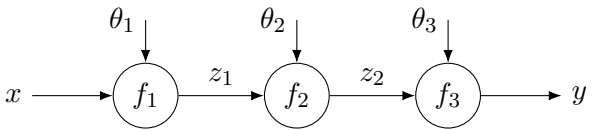



```python
class InvSqrtFcn(torch.autograd.Function):
    """Differentiable inverse square-root."""

    @staticmethod
    def forward(ctx, x):
        with torch.no_grad():
            y = 1.0 / torch.sqrt(x)

        # save state for backward pass
        ctx.save_for_backward(y)

        # return result
        return y

    @staticmethod
    def backward(ctx, dLdy):
        # check for None tensors
        if dLdy is None:
            return None

        # unpack cached tensors
        y = ctx.saved_tensors

        # compute and return gradients
        dLdx = None
        if ctx.needs_input_grad[0]:
            dLdx = -0.5 * torch.pow(y, 3.0) * dLdy
        return dLdx
```

Clone and Detach

Clone.

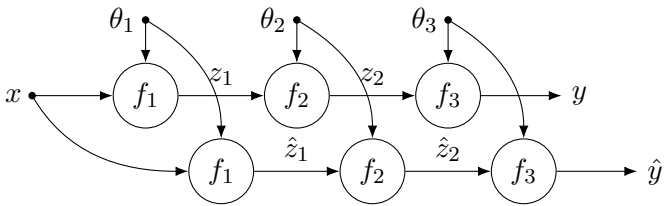
```
1 y_hat = y.clone() # creates copy of computation graph, gradients backpropagate
```



Clone and Detach

Clone.

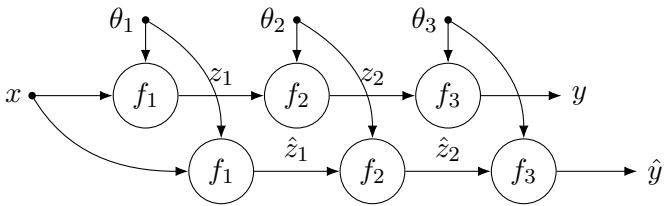
```
y_hat = y.clone() # creates copy of computation graph, gradients backpropagate
```



Clone and Detach

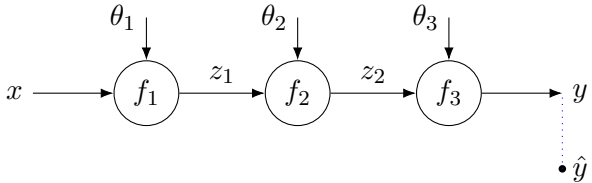
Clone.

```
y_hat = y.clone() # creates copy of computation graph, gradients backpropagate
```



Detach.

```
y_hat = y.detach() # shares memory with y but no gradients backpropagate
```

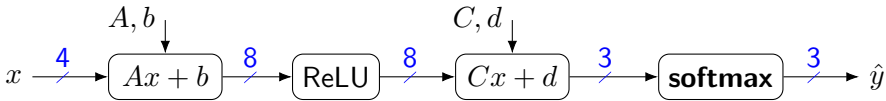


putting it all together

Iris Classification Example

- Iris dataset (Fisher, 1936)
  - 3 classes: Setosa, Versicolour, and Virginica
  - 4 features: sepal length, sepal width, petal length and petal width
  - 150 examples (50 per class)

- multi-layer perceptron classifier
  - 4 inputs, 8 hidden nodes (ReLU activation), 3 outputs (softmax)

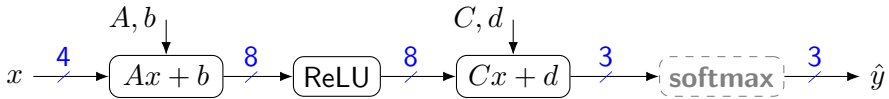


- gradient descent training with cross-entropy loss

$$\ell(\theta) = -\log\left[\textbf{softmax}(f(x;\theta))\right]_y, \qquad \theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$$

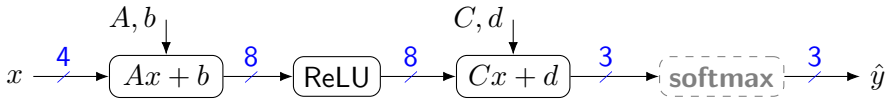
Defining the Model in PyTorch

- PyTorch's `nn.CrossEntropyLoss` function includes **softmax** so we can removed it from our model (Why does this not matter for prediction?)



Defining the Model in PyTorch

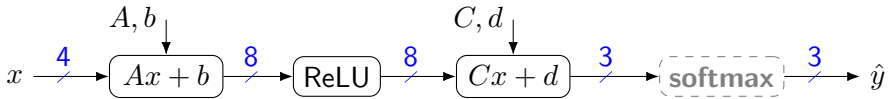
- PyTorch's `nn.CrossEntropyLoss` function includes **softmax** so we can removed it from our model (Why does this not matter for prediction?)




```
model = nn.Sequential(
            nn.Linear(4, 8, bias=True),
            nn.ReLU(True),
            nn.Linear(8, 3, bias=True)
        )
```

Defining the Model in PyTorch

- PyTorch's `nn.CrossEntropyLoss` function includes **softmax** so we can removed it from our model (Why does this not matter for prediction?)




```
model = nn.Sequential(
            nn.Linear(4, 8, bias=True),
            nn.ReLU(True),
            nn.Linear(8, 3, bias=True)
        )
```

```
class IrisMLP(nn.Module):
    """Multi-layer perceptron for Iris Dataset."""

    def __init__(self):
        super().__init__()

        self.layer1 = nn.Linear(4, 8, bias=True)
        self.layer2 = nn.ReLU(True)
        self.layer3 = nn.Linear(8, 3, bias=True)

    def forward(self, x):
        z1 = self.layer1(x)
        z2 = self.layer2(z1)
        y_hat = self.layer3(z2)

        return y_hat

model = IrisMLP()
```

Parameter Initialization

How do we initialize parameters A, b, C and d?

Parameter Initialization

How do we initialize parameters A, b, C and d?

- Good approach is to randomly draw i.i.d. from $\mathcal{N}(0, \sigma^2)$ or $\mathcal{U}(-\sigma, \sigma)$
- Biases are typically set to zero
- Golorot or Xavier initialization

$$\sigma = \sqrt{\frac{2}{n_i + n_o}}$$

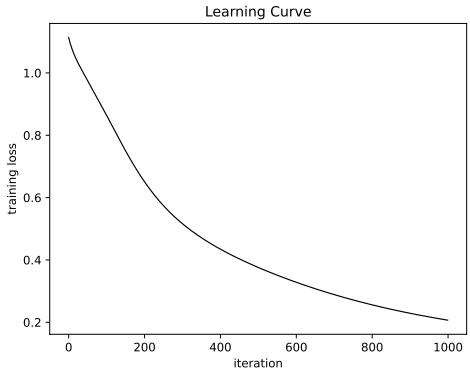
where n_i and n_o are the number of inputs and number of outputs, respectively.

- Kaiming initialization

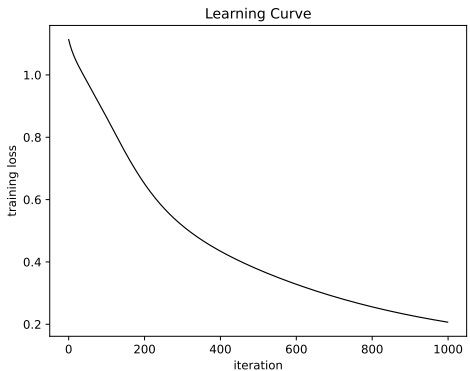
$$\sigma = \sqrt{\frac{2}{n_i}} \quad \text{or} \quad \sigma = \sqrt{\frac{2}{n_o}}$$

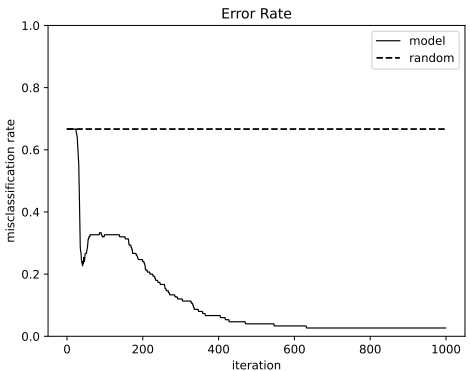
- Results in approximately constant feature variance throughout the network
- Parameters are initialized automatically in PyTorch when the model is created

Learning Curves



Learning Curves





Stochastic Gradient Descent (SGD)

- using the entire dataset to calculate the gradient is very expensive for big models
- many loss functions decompose over training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, so

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \, \ell(f(x^{(i)}; \theta), y^{(i)})$$

Stochastic Gradient Descent (SGD)

- using the entire dataset to calculate the gradient is very expensive for big models
- many loss functions decompose over training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, so

$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \, \ell(f(x^{(i)}; \theta), y^{(i)})$$

- SGD approximates the gradient on **mini-batches**, $\mathcal{I} \subseteq \{1, \ldots, N\}$

$$\widehat{\nabla_\theta L} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \nabla_\theta \, \ell(f(x^{(i)}; \theta), y^{(i)})$$

  - $|\mathcal{I}|$ is called the **batch size**

Stochastic Gradient Descent (SGD)

- using the entire dataset to calculate the gradient is very expensive for big models
- many loss functions decompose over training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$, so

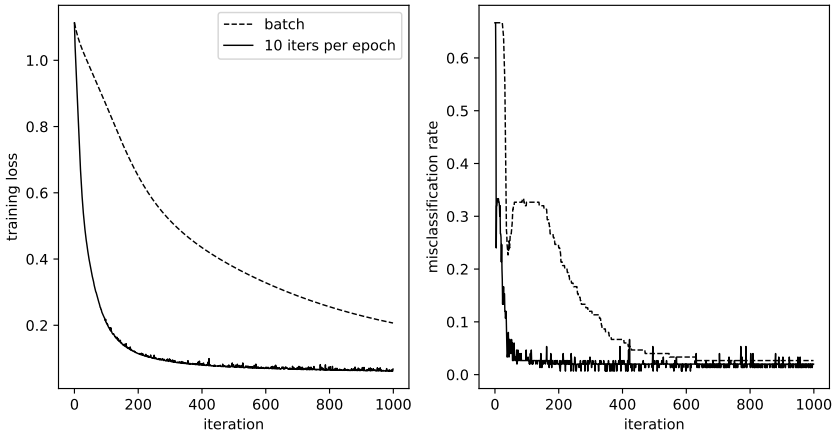
$$\nabla_\theta L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta \, \ell(f(x^{(i)}; \theta), y^{(i)})$$

- SGD approximates the gradient on **mini-batches**, $\mathcal{I} \subseteq \{1, \ldots, N\}$

$$\widehat{\nabla_\theta L} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \nabla_\theta \, \ell(f(x^{(i)}; \theta), y^{(i)})$$

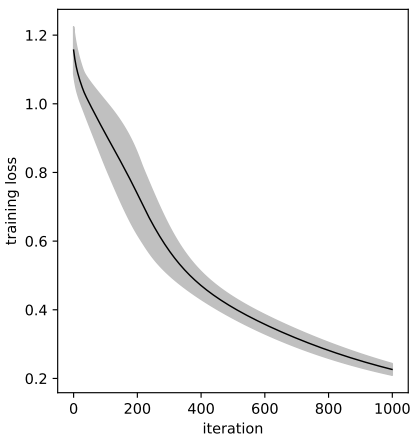
  - $|\mathcal{I}|$ is called the **batch size**
- under mild assumptions $E[\widehat{\nabla_\theta L}] = \nabla_\theta L$
- can be accumulated over multiple mini-batches for a better immediate estimate
- in practice we shuffle $[N]$ and iterate through adjacent fixed-length intervals
  - each gradient update step is called an **iteration**
  - once through the dataset is called an **epoch**

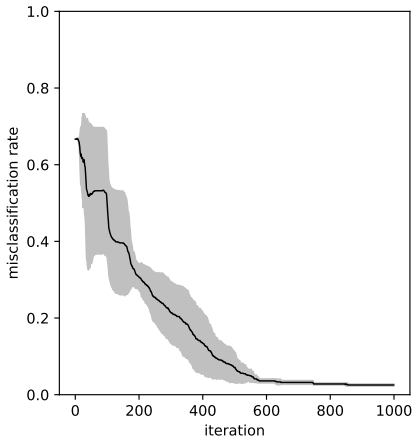
Learning Curves: SGD vs GD



Random Seeds

- deep learning models are non-convex in general so different initializations and mini-batch orderings may produce different results





(shows mean and one standard deviation over five runs)

Choosing the Best Model

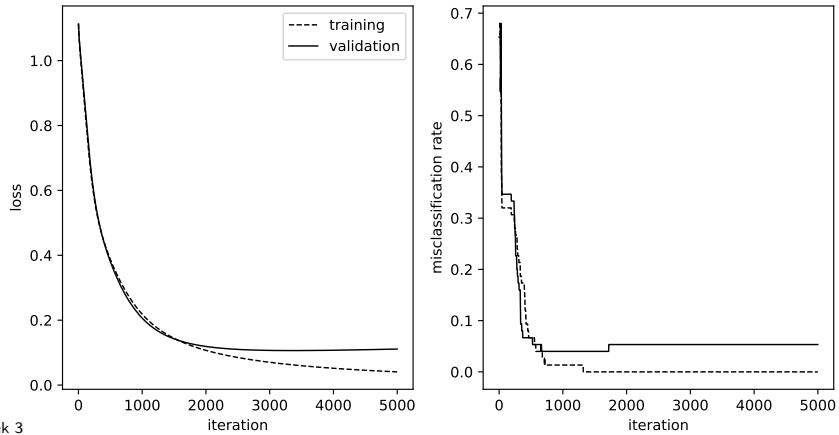
How do we select the best model to deploy?

Choosing the Best Model

- taking the model from the last iteration is not always the best
- standard practice is to break dataset into three disjoint subsets
  - **training set**: used for updating the model parameters
  - **validation set**: used for selecting the model to deploy
  - **test set**: used to evaluate generalization performance (ideally never seen)

Choosing the Best Model

- taking the model from the last iteration is not always the best
- standard practice is to break dataset into three disjoint subsets
  - **training set**: used for updating the model parameters
  - **validation set**: used for selecting the model to deploy
  - **test set**: used to evaluate generalization performance (ideally never seen)

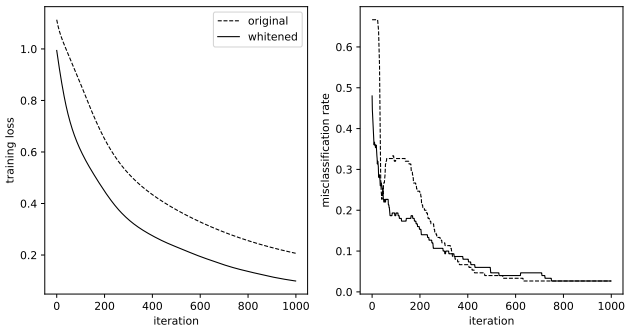


Improving Generalization Performance

- several mechanisms exist for improving generalization performance
  - regularization (standard machine learning approach)
    - early stopping (if validation set is not available)
  - data augmentation (add noise/perturbations to the data)
  - collect more real data (can be expensive)
  - large-scale generic **pre-training** followed by task-specific **finetuning**

Data Whitening

- normalizing data, $x = \Sigma^{-\frac{1}{2}}(x - \mu)$, often improves convergence and generalization



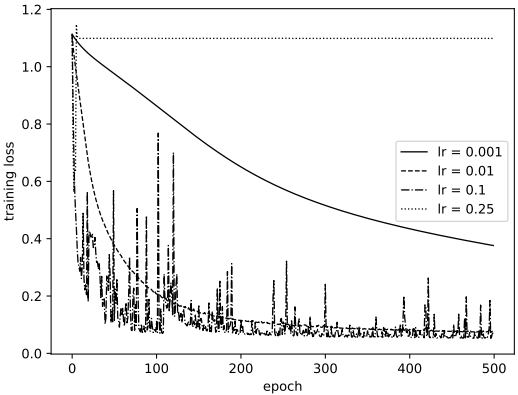
(caveat: learning rate rescaled based on average feature magnitude)

- on large scale data, normalize dimensions independently, $x_i = (x_i - \mu_i)/\sigma_i$
- applied layerwise in deep models using **batch normalization** layers[†]

[†]BatchNorm does a few other things, e.g., estimate linear transform and keep statistics for test time

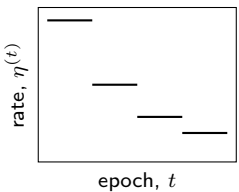
Learning Rate

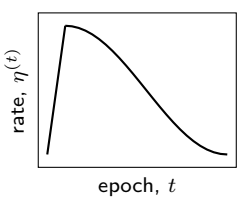
- the learning rate, η, is an example of a **hyperparameter**
- it can have a big influence on performance and convergence rate



Learning Rate Schedules

- we don't need to use the same learning rate each epoch
- popular learning rate schedules include
  - **constant**: $\eta^{(t)} = \eta_0$
  - **linear**: $\eta^{(t)} = \eta_{\text{init}} + \frac{t}{t_{\text{max}}} (\eta_{\text{final}} - \eta_{\text{init}})$
  - **step**: $\eta^{(t)} = \begin{cases} \gamma \eta^{(t-1)}, & \text{if } t \,\%\, t_{\text{step}} = 0 \\ \eta^{(t-1)}, & \text{otherwise} \end{cases}$
  - **cosine**: $\eta^{(t)} = \eta_{\text{min}} + \frac{1}{2}(\eta_{\text{max}} - \eta_{\text{min}}) \left(1 + \cos\left(\frac{\pi t}{t_{\text{period}}}\right)\right)$
- combinations, e.g., linear warm-up with cosine decay
- a jump in training loss often accompanies a step change in learning rate





Momentum and SGD Variants

- stochastic gradient can be slow even with a good learning rate schedule
- prompted researchers to explore alternative pseudo-second-order schemes
- one popular method is to add **momentum**,

$$g^{(t)} = \nabla L(\theta^{(t)}) + \mu g^{(t-1)}$$
$$\theta^{(t+1)} = \theta^{(t)} - \eta g^{(t)}$$

Momentum and SGD Variants

- stochastic gradient can be slow even with a good learning rate schedule
- prompted researchers to explore alternative pseudo-second-order schemes
- one popular method is to add **momentum**,

$$g^{(t)} = \nabla L(\theta^{(t)}) + \mu g^{(t-1)}$$
$$\theta^{(t+1)} = \theta^{(t)} - \eta g^{(t)}$$

- other popular methods include AdaGrad, Adam and AdamW (next)
- need to play around to see what works best for your problem
  - start with a method and schedule that had been demonstrated to work well for similar tasks/architectures and then adapt hyper-parameters

AdaGrad

(Duchi et al., 2011)

- view gradient descent as a first-order proximal method

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_2^2 \right\} \\ &= \theta^{(t)} - \eta_t \nabla L(\theta^{(t)})\end{aligned}$$

AdaGrad

(Duchi et al., 2011)

- view gradient descent as a first-order proximal method

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_2^2 \right\} \\ &= \theta^{(t)} - \eta_t \nabla L(\theta^{(t)})\end{aligned}$$

- adapt the step size (geometry) for different features to increase influence of rare but informative features

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_B^2 \right\} \\ &= \theta^{(t)} - \eta_t B^{-1} \nabla L(\theta^{(t)})\end{aligned}$$

AdaGrad

(Duchi et al., 2011)

- view gradient descent as a first-order proximal method

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_2^2 \right\} \\ &= \theta^{(t)} - \eta_t \nabla L(\theta^{(t)})\end{aligned}$$

- adapt the step size (geometry) for different features to increase influence of rare but informative features

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_B^2 \right\} \\ &= \theta^{(t)} - \eta_t B^{-1} \nabla L(\theta^{(t)})\end{aligned}$$

- use previous gradients to estimate B as diagonal matrix

$$G^{(t)} = \left(\sum_{k=1}^{t} \mathbf{diag}\big(\nabla L(\theta^{(k)})\big)^2 + \epsilon I \right)^{1/2}$$

AdaGrad

(Duchi et al., 2011)

- view gradient descent as a first-order proximal method

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_2^2 \right\} \\ &= \theta^{(t)} - \eta_t \nabla L(\theta^{(t)})\end{aligned}$$

- adapt the step size (geometry) for different features to increase influence of rare but informative features

$$\begin{aligned}\theta^{(t+1)} &= \arg\min_{\theta} \left\{ \langle \nabla L(\theta^{(t)}), \theta \rangle + \frac{1}{2\eta_t} \|\theta - \theta^{(t)}\|_B^2 \right\} \\ &= \theta^{(t)} - \eta_t B^{-1} \nabla L(\theta^{(t)})\end{aligned}$$

- use previous gradients to estimate B as diagonal matrix

$$G^{(t)} = \left(\sum_{k=1}^{t} \mathbf{diag}\big(\nabla L(\theta^{(k)})\big)^2 + \epsilon I \right)^{1/2}$$

- weakness: $G^{(t)}$ keeps growing so learning rate becomes infinitesimally small

Adam

(Kingma and Ba, 2014)

- maintains exponentially decaying averages of past gradients and gradients-squared

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1)\nabla L(\theta^{(t)})$$
$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2)\nabla L(\theta^{(t)})^2$$

Adam

(Kingma and Ba, 2014)

- maintains exponentially decaying averages of past gradients and gradients-squared

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1)\nabla L(\theta^{(t)})$$
$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2)\nabla L(\theta^{(t)})^2$$

- compensate for bias (c.f. unbiased variance calculations)

$$\hat{m}^{(t)} = \frac{1}{1 - \beta_1^t} m^{(t)} \text{ and } \hat{v}^{(t)} = \frac{1}{1 - \beta_2^t} v^{(t)}$$

Adam

(Kingma and Ba, 2014)

- maintains exponentially decaying averages of past gradients and gradients-squared

$$m^{(t)} = \beta_1 m^{(t-1)} + (1 - \beta_1)\nabla L(\theta^{(t)})$$
$$v^{(t)} = \beta_2 v^{(t-1)} + (1 - \beta_2)\nabla L(\theta^{(t)})^2$$

- compensate for bias (c.f. unbiased variance calculations)

$$\hat{m}^{(t)} = \frac{1}{1 - \beta_1^t} m^{(t)} \text{ and } \hat{v}^{(t)} = \frac{1}{1 - \beta_2^t} v^{(t)}$$

- apply update rule

$$\theta^{(t+1)} = \theta^{(t)} - \eta \, \mathbf{diag}\left(\hat{v}^{(t)} + \epsilon\right)^{-1/2} \hat{m}^{(t)}$$

AdamW

(Loshchilov and Hutter, 2019)

- many other variants of update rules
- for example, Adam with weight decay

$$\theta^{(t+1)} = \theta^{(t)} - \eta \, \mathbf{diag}\left(\hat{v}^{(t)} + \epsilon\right)^{-1/2} \left(\hat{m}^{(t)} + w\theta^{(t)}\right)$$

- empirically works better than applying ℓ_2 regularization on $m^{(t)}$
- de facto standard update rule (for now)

Momentum and SGD Variants Learning Curves